

A self-maintaining AI solution for chemical-free weeding robots.

End-to-end MLOps: specification, serving, CI/CD, drift monitoring and automated retraining

June 2026

The MLOps loop matters more than the model

1

Herbicide bans and labour shortages make autonomous weeding a compliance issue, not a gadget

- France's Loi Labbe already bans synthetic herbicides in public green spaces; organic farming forbids them outright
- Manual weeding is the most labour-intensive task in chemical-free grounds care, exactly where labour is scarcest

2

A deliberately simple classifier proves a production-grade platform

- ResNet18 transfer learning reaches 0.89 macro F1 with about 400 images and a 4 minute CPU training run
- Serving degrades gracefully, every run is tracked in MLflow, every commit is tested, built and validated by CI

3

Drift detection closes the loop with gated, automated retraining

- One PSI engine feeds both the Grafana dashboard and the retrain decision, so they can never disagree
- A promotion gate ships a candidate only if it clears 0.85 macro F1 and does not regress against the live model

Agenda

Specifications and business case

ML solution and serving

CI/CD, monitoring and retraining

Weeding is turning into a compliance problem that hand labour can no longer absorb.

0

synthetic herbicides allowed in French public green spaces under Loi Labbe

#1

cost line in chemical-free grounds care is manual weeding labour

2

classes the robot must never confuse: the weed and the crop it protects

24/7

autonomous operation only pays if the perception model stays reliable unattended

- **The product:** GreenGuard robots remove weeds mechanically; the robot acts only when the camera is sure it sees a weed
- **The economics:** every percentage point of precision protects the asset; every point of recall is herbicide-free weeding that actually worked
- **Why MLOps is the product:** fields, seasons and lighting drift continuously; a model that cannot watch itself and retrain silently decays into a crop-cutting liability

Every business outcome maps to one ML metric and one live monitoring signal; nothing is tracked for decoration.

If a metric moves, someone knows which business promise is at risk.

Business outcome	ML metric	Live signal
Never cut the crop	Per-class precision, confusion matrix	Prediction mix, drift flag, retrain events
Do not miss weeds	Macro F1, recall on the weed class	Release gate at 0.85; drift flag in Grafana
Real-time operation on a moving robot	Inference latency	p90 latency vs the 500 ms target in Prometheus
Sustained accuracy across sites and seasons	Drift magnitude, retrain cadence	Dataset drift flag, drift share, retrain runs
Low maintenance cost	Automation coverage	Green CI/CD, drift-triggered retraining

Source: docs/SPECIFICATION.md, business outcome to metric table

Seven functional requirements, each with a written acceptance criterion the jury can check in the repo.

Requirement	Acceptance criterion	Where it is verified
Data ingestion and preprocessing	Weekly Airflow DAG runs end to end	dandelion_data_pipeline
Reproducible training	Fixed seed, pinned deps, every run tracked	MLflow run history
Prediction API	Class plus probabilities; 400 on a non-image	POST /predict, tests/test_api.py
Health, version, metrics	Graceful startup; model metadata exposed	/health, /version, /metrics
Web UI	Upload an image, see class and confidence	Streamlit app
Drift detection	Per-feature PSI plus a dataset drift flag	Grafana drift dashboard, drift report
Automated retraining	Scheduled and drift-triggered, gated	dandelion_retrain_pipeline

Source: docs/SPECIFICATION.md; deliverables/cahier_des_charges.docx

CHAPTER 2

ML solution and serving

400 images, a 4 minute training run, and an API that never goes down with its model

With 400 images, the architecture decision is the whole ML decision: transfer learning or nothing.

Too little data to learn vision from scratch; exactly enough to fine-tune it.

	ResNet18 transfer learning (production)	4-block CNN from scratch (baseline)
Representation	ImageNet features reused; only the head retrains	Must learn edges and texture from 400 images
Data appetite	Hundreds of images suffice	Typically needs orders of magnitude more
Cost	About 4 minutes on CPU	Cheaper per epoch, far weaker ceiling
Role here	Serves production traffic	Kept in the codebase for offline ablation
- Augmentation on the training split only: random resized crop, horizontal flip, colour jitter; 80/20 split at seed 42		

Source: notebooks/02_model_selection.ipynb, models/model.py

The production model clears its release gate with margin, and the gate is enforced in code, not in a slide.

Validation metrics, ResNet18 production model, score, 0 to 1



4 min

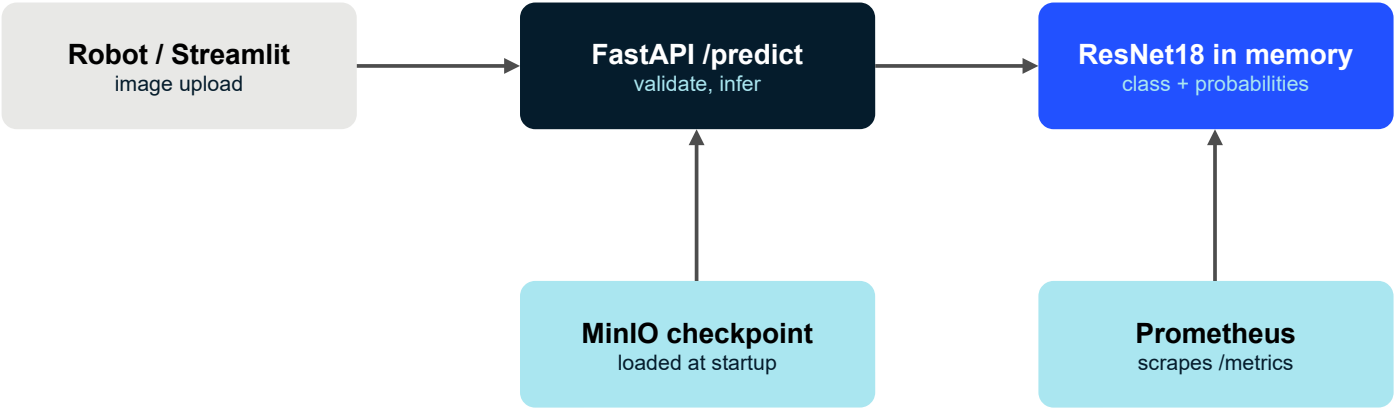
full training run on CPU;
no GPU budget needed

100%

of runs tracked in MLflow:
params, metrics, artifacts

- Best checkpoint selected on validation accuracy; architecture and class names travel inside the checkpoint, so serving can never load the wrong shape

The API treats the model as cargo: if the model is missing, serving stays up and says so.



Production posture

- **Stateless** container behind a Kubernetes Service with liveness and readiness probes
- **Scalable** horizontally; the model loads from MinIO, never trains in the serving path

Endpoint	Returns	Failure behaviour
POST /predict	Class, confidence, per-class probabilities	400 invalid image, 503 while loading
GET /health, /version	Readiness, classes, architecture, source	Reports initializing until ready
GET /metrics	Counters, latency histogram, drift gauges	Gauges skipped if no drift report

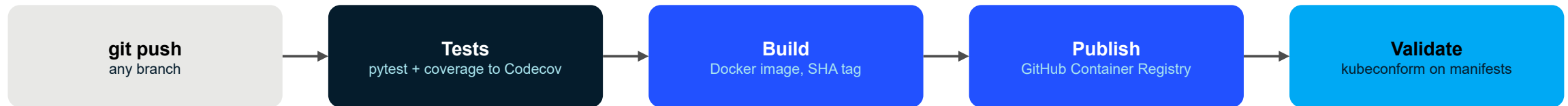
Source: app/api/main.py; tests/test_api.py; k8s/minikube-manifest.yaml

CHAPTER 3

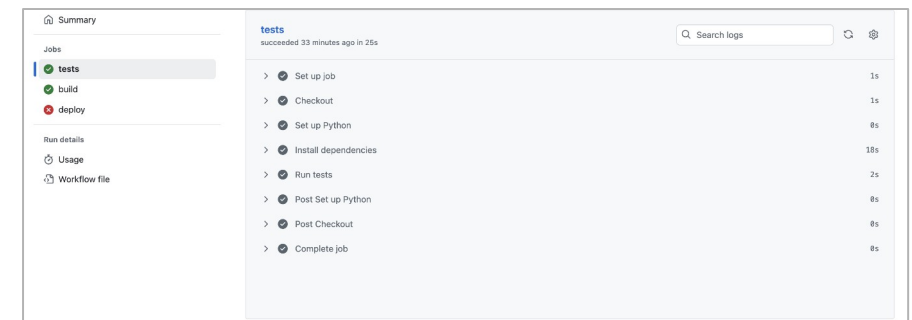
CI/CD, monitoring and retraining

Every commit tested and shipped; the data watched; the model able to replace itself

From git push to a deployable, schema-validated image with no human in the loop.



- **Deterministic over flaky:** offline manifest validation replaced a live Minikube rollout that failed randomly on hosted runners; the same manifests deploy to Minikube locally
- **Traceable:** SHA-tagged images mean any running pod maps to exactly one commit
- **Hardened:** manifests carry liveness and readiness probes and resource limits on all four deployments



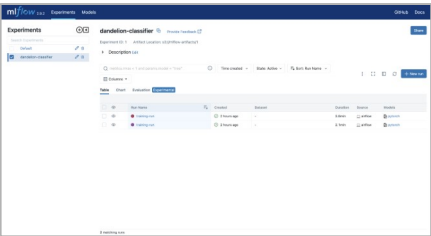
CI run: tests, build, deploy validation

Source: [.github/workflows/ci_cd.yml](https://github.com/GoogleCloudPlatform/workflows/ci_cd.yml)

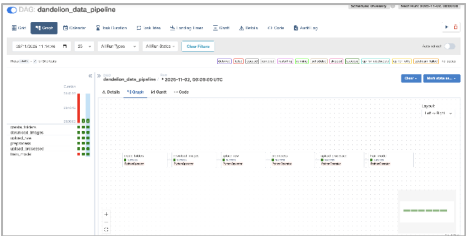
One Grafana stack answers three questions: is serving healthy, are the pipelines running, is the data still the same.



Classifier dashboard: total predictions and p90 latency vs the 500 ms target

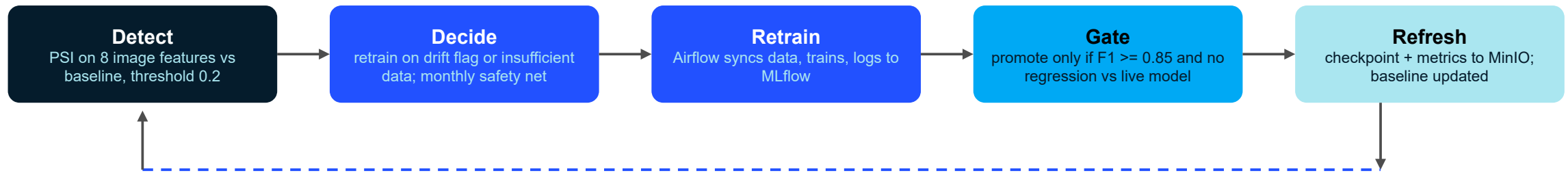


MLflow: every training run tracked
Source: monitoring/grafana/dashboards; Prometheus scrapes API and Airflow every 15 s; drift gauges served by the API



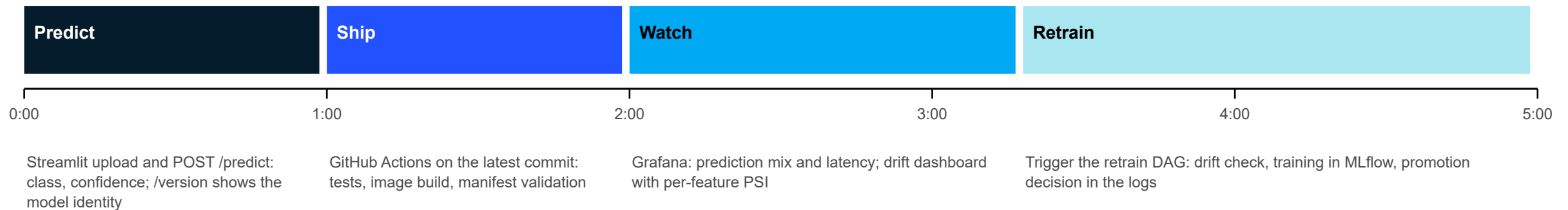
Airflow: data and retrain DAGs

The model watches its own inputs and replaces itself only when the replacement is provably better.



- **One definition of drift:** the same PSI engine feeds the Grafana dashboard and the retrain gate, so monitoring and automation can never disagree
- **No silent failures:** the drift check never breaks the DAG; the first run seeds the baseline instead of failing
- **No accidental regressions:** the promotion gate compares the candidate to the live model before anything is uploaded

Five minutes from a live prediction to a retraining decision, on the running stack.



- demo.sh runs the whole sequence on one command; docs/LOOM_SCRIPT.md scripts the narration shot by shot

Known limits each carry a named next increment; none of them blocks the loop the thesis demonstrates.

Current limit	Why it is acceptable today	Next increment
Binary classifier on an easy dataset	The thesis demonstrates the loop, not the model	Multi-class weeds, harder field imagery
Drift measured on image statistics	Interpretable, cheap, catches lighting and season shifts	Embedding-based drift for semantic change
Retraining fully automated	Gated by the F1 floor plus regression check	Human approval step and retrain cooldown
No alerting rules yet	Dashboards reviewed during weekly ops	Prometheus alerts wired to the retrain webhook

Source: docs/MODEL_CARD.md limitations; docs/DEFENSE.md

From a working model to a self-maintaining AI product.

- Live demo: prediction API, CI/CD run, drift dashboard, retraining DAG
- Code, tests, coverage and documentation public on GitHub
- Cahier des charges and model card in the deliverables folder